

TEAM MEMBERS

- ◆ Abhiroop
- ◆ Mehrish
- ◆ Hasini
- ◆ Ronith
- ◆ Rushita

RELEASE 1

3D Print Cost Estimator

Five Fingers Innovative Solutions

Team No. 8 · 3DPrintVision

Agenda

01

Project Overview

Goals, problem statement & stakeholders

02

System Architecture

Tech stack, components & data flow

03

Features Built

All features delivered in Release 1

04

Demo Flow

End-to-end user journey walkthrough

05

Printability Analysis

Automated model quality checks

06

Cost Estimation Engine

How pricing is calculated

07

What's Next

Upcoming features for Release 2

What We're Building

The Problem

3D printing cost estimation is manual, slow and error-prone. Print shops spend hours pre-checking models and quoting jobs — leading to delays and human mistakes.

Self-Service Upload

Users upload STL/OBJ/STEP files via drag & drop — no manual handoff needed

Automated Slicing

Trimesh-based slicer generates G-code with full parameter control

Instant Cost Estimate

Material weight × price + machine time rate → accurate INR quote in seconds

Printability Checks

Automated detection of thin walls, overhangs, and non-watertight meshes

Admin Controls

Configurable materials, pricing rules, machine rates and failure factors

3D Visualisation

Interactive WebGL viewer with solid, wireframe and X-ray modes

Stakeholders & Scope

User Roles

Customer

Uploads models, views cost estimates, downloads quotes

Admin

Manages materials, pricing config, views failure logs

In Scope — Release 1

STL file upload with drag & drop

Server-side validation & sanitisation

Geometric analysis (volume, manifold, overhangs)

Trimesh-based slicing engine

Cost estimation with configurable pricing

WebGL 3D viewer with orbit controls

User auth (register / login)

Admin panel — materials & config

G-code layer preview with animation

Downloadable PDF quote

Technology Stack

FRONTEND

React 18 + TypeScript

- › Vite 4 build tool
- › Material UI 5 components
- › Three.js r150 — 3D viewer
- › React Dropzone — file upload
- › Axios — API communication

BACKEND

Python 3.11 + FastAPI

- › SQLAlchemy ORM
- › Trimesh — slicing engine
- › NumPy — geometry analysis
- › python-jose — JWT auth
- › Passlib + bcrypt — passwords

DATABASE

PostgreSQL 15

- › Jobs, users, materials tables
- › SlicingResult with JSONB params
- › AdminConfig key-value store
- › FailureLog for error tracking
- › Alembic migrations

INFRA

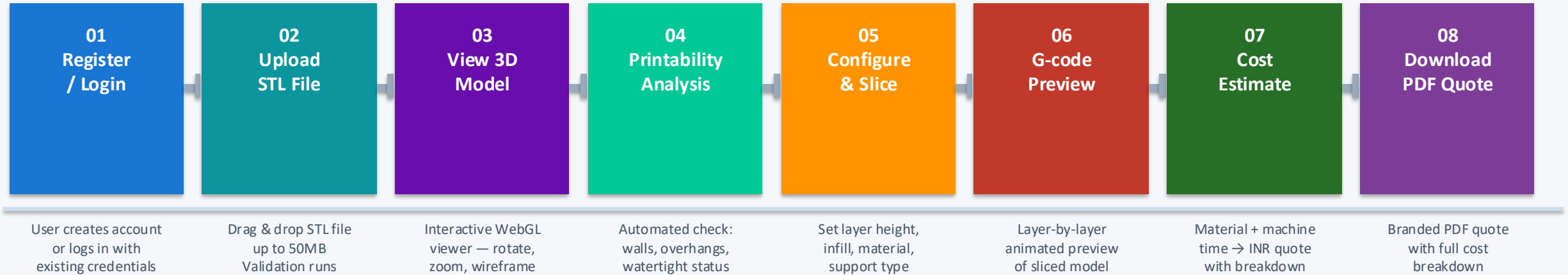
Docker + Compose

- › 4 containers — frontend, backend, postgres, cura-engine
- › Hot-reload dev environment
- › Volume mounts for uploads/gcode
- › Health checks on all services

Component Architecture



End-to-End User Journey



⚡ Each step is accessible from the Job Detail page — users can re-slice with different parameters and recalculate costs any time.

What We Built

File Upload & Validation

Drag & drop STL files up to 50MB with server-side extension and integrity validation

Interactive 3D Viewer

Three.js WebGL viewer — solid, wireframe, X-ray modes. Model centred on grid with orbit controls

Trimesh Slicing Engine

Pure Python slicer using trimesh cross-sections — no CuraEngine binary required. Full parameter control

G-code Layer Preview

Animated layer-by-layer visualisation with play/pause/speed controls and colour-coded layers

Printability Analysis

Automatic checks for thin walls, overhangs $>45^\circ$, non-watertight meshes with actionable guidance

Cost Estimation Engine

Material weight $\times$ cost/gram + machine time $\times$ hourly rate with waste and failure overhead factors

Downloadable PDF Quote

Branded quote PDF with full cost breakdown, job details and 7-day validity — generated in-browser

User Authentication

JWT-based login/register. Role-based access — customer vs admin with protected routes

Admin Panel

Manage materials (PLA/ABS/PETG/TPU), pricing config, machine rates, and failure log viewer

3D Model Viewer

3D Viewer | Solid | Wireframe | X-Ray | Reset View



STL model
centred on grid

Implementation Details

Library

Three.js r150 with OrbitControls

View Modes

Solid · Wireframe · X-Ray (0.3 opacity)

Centering Fix

Bounding box centred on X/Z axes;
bottom placed on grid Y=0

Camera

$\text{maxDim} \times 1.5$ diagonal offset ensures
full model always in view

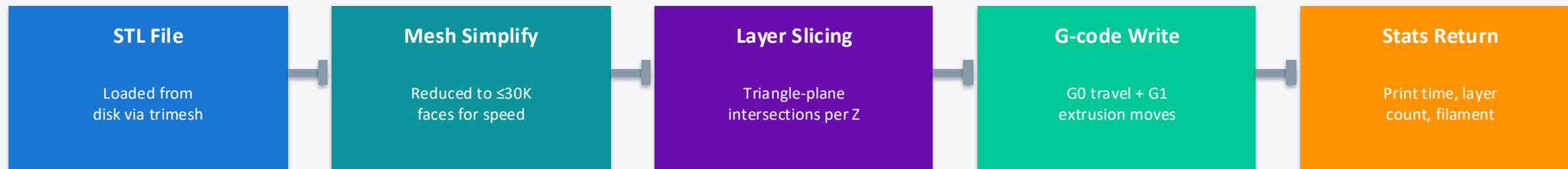
Model Info

Dimensions (mm), triangle count
displayed after load

Cleanup

Geometry & material disposed on
unmount to prevent memory leaks

Slicing Engine



Configurable Slicing Parameters

Material:	PLA · ABS · PETG · TPU	Layer Height:	0.05mm → 0.4mm
Infill Density:	0% → 100%	Infill Pattern:	Grid · Lines · Triangles · Cubic · Gyroid
Wall Thickness:	Configurable in mm	Support Type:	None · Touching Buildplate · Everywhere
Print Speed:	mm/s configurable	Bed Adhesion:	None · Skirt · Brim · Raft
Nozzle Temp:	°C configurable	Bed Temp:	°C configurable

Printability Analysis

What Gets Checked

WARNING

Watertight Check

Verifies mesh has no holes or non-manifold edges.
Essential for slicing — open meshes fail to slice correctly.

WARNING

Thin Walls

Flags any dimension $< 0.8\text{mm}$.
Thin features may not print or will be very fragile.

INFO

Overhang Detection

Identifies faces with angle $> 45^\circ$ from vertical.
These need support structures to print successfully.

INFO

Model Dimensions

Reports $X \times Y \times Z$ bounding box in mm.
Helps confirm the model fits the printer build volume.

INFO

Volume & Area

Calculates volume (cm^3) and surface area (cm^2).
Used directly in the cost estimation calculation.

INFO

Triangle Count

Reports mesh resolution.
Very high polygon counts may slow slicing.

Example Output

✓ No issues found — model is print-ready

 2 issues detected — review before printing

Cost Estimation Engine

Core Formula

Total Cost = (Material Cost + Support Cost) × Waste Factor × Failure Factor + Machine Time Cost

Material Cost

$\text{material_weight_g} \times \text{cost_per_gram}$
(from materials table)

PLA: ₹0.025/g · ABS: ₹0.028/g
PETG: ₹0.032/g · TPU: ₹0.045/g

Support Cost

$\text{support_weight_g} \times \text{cost_per_gram}$
(10% of model volume if enabled)

Only added if support
structures are enabled

Machine Time

$\text{print_time_hours} \times \text{machine_hourly_rate}$
(from admin config)

Default: ₹500/hour
Configurable by admin

Waste Factor

Multiplier applied to material cost
(accounts for failed prints, waste)

Default: 1.25×
(25% overhead)

Failure Factor

Second multiplier on wasted material
(accounts for print failure risk)

Default: 1.25×
(25% failure risk)

Database Schema

users

id PK

username unique

password_hash

email

role (customer/admin)

created_at

materials

id PK

name unique

density_g_cm3

cost_per_gram

created_at

jobs

job_id UUID PK

user_id FK→users

filename

original_filename

file_path

file_size

admin_config

key PK

value

description

updated_at

slicing_results

id PK

job_id FK→jobs

gcode_path

print_time_seconds

material_weight_grams

layer_count

failure_logs

id PK

job_id FK→jobs

error_type

error_message

stack_trace

created_at

API Endpoints

POST	/api/auth/register Register a new user account	POST	/api/auth/login Login and receive JWT token
GET	/api/auth/me Get current user info	POST	/api/upload Upload STL file, create job
GET	/api/jobs List all jobs for current user	GET	/api/job/{job_id} Get job details + slicing result
POST	/api/slice/{job_id} Start slicing with parameters	GET	/api/files/{job_id}/model.stl Download original STL file
GET	/api/files/{job_id}/output.gcode Download generated G-code	POST	/api/estimate/{job_id} Calculate cost estimate
GET	/api/analyze/{job_id} Run printability analysis	GET	/api/admin/materials List all materials
POST	/api/admin/materials Create new material	PUT	/api/admin/materials/{id} Update material pricing
DELETE	/api/admin/materials/{id} Remove a material	GET	/api/admin/config Get pricing configuration
PUT	/api/admin/config Update config value	GET	/api/admin/logs View failure logs

Challenges & How We Solved Them

3D Model Not Centred

Problem: After loading the STL, the model appeared in the corner of the viewer instead of the centre of the grid

Solution: Fixed by computing the bounding box centre after loading, offsetting X/Z axes explicitly, and lifting Y to sit on the grid. Camera distance based on $\text{maxDim} \times 1.5$ for consistent framing.

G-code Preview Off-Grid

Problem: Post-slicing G-code preview showed the model offset to one corner because the slicer offsets coordinates to X=100, Y=100 (printer bed origin)

Solution: Applied a bounding box centring pass on all parsed G-code points before building Three.js LineSegments, zeroing X/Z while setting Y floor to zero.

CuraEngine Missing

Problem: docker compose up failed because the cura-engine folder was gitignored — no Dockerfile present

Solution: Created a minimal Alpine-based placeholder Dockerfile for the cura-engine service. The actual slicing uses Python trimesh, so no real CuraEngine binary is needed.

Cost Estimate Crashing

Problem: Calculate Cost Estimate returned 500 error — backend crashed looking for PLA in the materials table

Solution: Database was empty after fresh setup. Created seed_data.py to populate materials and admin_config tables. Must be run once after docker compose up --build.

What's Next — Release 2

Re-slice

UX

Allow users to change slicing parameters and re-slice an already-sliced job

OBJ / STEP Support

Scope

Extend file upload and slicing pipeline to handle OBJ and STEP formats

Saved Quotes

Scope

Let users save, name and retrieve past cost estimates from their dashboard

Email Notifications

Integration

Mailtrap integration fully wired — send on upload, slice complete, and failure

Section View

3D Viewer

Interactive cross-section slice plane in the 3D viewer (requirements spec)

Performance

NFR

≥90% of models produce cost estimate within 10s (per NFR requirements)

Thank You

Five Fingers Innovative Solutions · Team No. 8

Abhiroop · Mehrish · Hasini · Ronith · Rushita

3DPrintVision · Release 1 Presentation

Questions?